Government Engineering College Kishanganj
Computer Networks Lab
Assignment 7: UDP Sockets

In this assignment, you will implement a simple application over UDP sockets in C, and will write two programs: udp_server.c and udp_client.c. Your server will implement a personalized UDP service, whose behavior is determined by your Roll Number. The client will communicate with this server using a binary/text protocol you design.

1. Port Number

Your server must bind to a UDP port computed from your roll number:

```
PORT = 30000 + (last 3 digits of your roll number)
```

Examples:

- Roll = 24105003 → last 3 digits = 003 → PORT = 30003
- Roll = 24105120 → last 3 digits = 120 → PORT = 30120

You must print the port number when the server starts, e.g.:

```
Server running on port 30120
```

2. Service Type

Your server implements ONE of the following service types based on your roll number:

```
Let d = (sum of digits of the last three digits of your roll number) mod 3
```

Choose the service according to d:

- d = 0 → Type A: Text Transformer
- d = 1 → Type B: Integer List Processor
- d = 2 → Type C: String Statistics

You must write in a comment at the top of your code:

```
// Roll: <your roll> Service Type: <A/B/C/D/E> d = <value>
```

a. Type A: Text Transformer

Client sends a request:

```
COMMAND|TEXT
```

where COMMAND is one of:

- UP – convert TEXT to uppercase
- LOW – convert TEXT to lowercase
- REV – reverse TEXT

Server responds with transformed text.

Example:

```
Request: UP|Hello world
Response: HELLO WORLD
```

b. Type B: Integer List Processor

Client sends:

```
OP|n|a1 a2 a3 ... an
```

where: OP is one of SUM, MAX, MIN, AVG

- n is number of integers
- a1...an are integers separated by spaces

The server responds with the result as a number (for AVG, sent as a float with 2 decimal places).

Example:

```
Request: SUM|5|1 2 3 4 5
Response: 15
```

c. Type C: String Statistics

Client sends a TEXT string (no | needed). Server responds with:

```
LEN=<len>;VOWELS=<count>;CONSONANTS=<count>;DIGITS=<count>
```

Example:

```
Request: Hello123
Response: LEN=8;VOWELS=2;CONSONANTS=3;DIGITS=3
```

3. Message Format

Every request from the client must begin with your roll number as a prefix, followed by #.

Example (Roll: 24105003):

```
24105003#UP|Hello world
24105003#SUM|5|1 2 3 4 5
24105003#QUICKHELLO
```

The server must check that the roll prefix matches the expected roll number (hard-coded in the server). If it does NOT match, respond with:

```
ERROR: Unauthorized client
```

and ignore the rest of the message.

4. Server Behavior

- The server must:
    - Create a UDP socket using socket()
    - Bind to the port calculated from your roll using bind()
    - Continuously call recvfrom() in a loop to receive requests.
    - Process each request and generate a reply.
    - Use sendto() to send the reply back to the client.
    - Print simple logs on the server side:
        - Client address (IP + port)
        - Received message (after removing roll prefix)
        - Sent response
- The server should exit cleanly when it receives a special command:
    - ROLL#EXIT (e.g., 24105003#EXIT).

5. Client Behavior

- The client must:
    - Take server IP address and roll number as command-line arguments:

      `./udp_client <server_ip> <roll>`
    - Create a UDP socket.
    - Repeatedly:
        - Read a line from the user (e.g., UP|Hello world).
        - Automatically prepend the roll prefix (<roll>#).
        - Send the full message with sendto().
        - Wait for and print the response from the server (recvfrom()).
    - On EOF or when the user types EXIT, the client must send the exit command and then terminate.